## Correction

Une **file de priorité** est une structure de données permettant les opérations suivantes :

- Ajouter un élément à la structure
- Extraire l'élément ayant la valeur maximale (renvoyer sa valeur et la supprimer de la structure)

On suppose que les éléments que l'on stocke dans la structure sont des entiers.

# 1 - Implémentation par un tableau

On propose une première implémentation de file de priorité :

- On utilise une structure de tableau (de taille suffisamment grande).
- Pour ajouter un élément, on le place en dernière position à la suite des valeurs déjà présentes.

1. Décrivez l'algorithme d'extraction de la valeur maximale. Quels sont alors les coûts en pire cas des opérations d'ajout et d'extraction, en fonction du nombre $n$ d'éléments présents ?

```
Pour extraire la valeur maximale, on parcourt les éléments du tableau dans
l'ordre, en retenant à chaque étape l'indice du plus grand élément rencontré.
Pour supprimer la valeur trouvée, on l'échange avec la dernière valeur, et on
diminue la taille logique de 1.

Le coût de l'extraction de la valeur maximale est O(n), car elle nécessite un
parcours du tableau. La suppression en elle-même s'effectue en temps constant
O(1).

L'opération d'ajout coûte un temps constant, en supposant que la taille du
tableau soit suffisante.
```

On propose un algorithme de tri basé sur la structure de file de priorité :

> **Tri par file de priorité**
> Pour trier un tableau $T$ composé de $n$ éléments :
>
> - On insère les $n$ éléments dans une file de priorité initialement vide.
> - On réalise $n$ extractions de la valeur max, et on insère ces valeurs dans le tableau $T$ en partant de la dernière position.

2. Avec l'implémentation actuelle de la file de priorité, à quel algorithme de tri classique correspond l'algorithme décrit ci-dessus ?
   Quel est le coût en pire cas de cet algorithme en fonction de $n$ ?

```
Avec l'implémentation actuelle, l'algorithme correspond au tri par sélection.

Pour un tableau de taille n, son coût est quadratique, O(n^2).
En effet, l'algorithme effectue une opération d'extraction du max qui coûte
tour à tour n, n-1, n-2, ..., 1.
Le coût total est donc 1 + 2 + ... + n-1 + n
    = n * (n + 1) / 2
    = O(n^2).
```

3. On souhaite pouvoir réaliser l'extraction de la valeur maximale en temps $O(1)$, en utilisant toujours une implémentation via un tableau. Proposez une **nouvelle stratégie** d'ajout et d'extraction pour garantir cette complexité. Donnez, en justifiant, la complexité des deux opérations.

**Ajout** d'un élément :

```
On maintient une structure de tableau trié : on ajoute notre élément à la
bonne position, en cherchant sa position (par exemple par une recherche
dichotomique, en O(log n)) et en décalant les éléments suivants vers la
droite.

Le coût total de l'opération d'ajout est donc linéaire, O(n).
```

**Extraction** de la valeur max :

```
Comme le tableau est supposé trié, sa valeur maximale est en dernière
position. L'extraction nécessite simplement de décrémenter la taille logique
du tableau.
Son coût est donc constant, O(1).
```

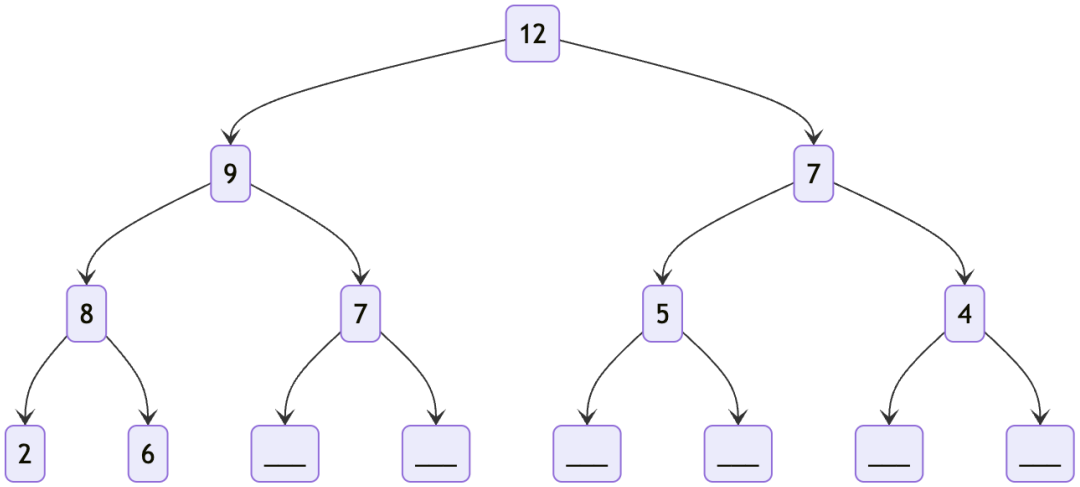
## 2 - Implémentation par un tas binaire

Nous allons proposer une implémentation plus performante des files de priorité, via une structure de **tas binaire**, qui est une variété d'arbre binaire.

Pour implémenter les arbres binaires, nous allons utiliser la représentation en mémoire suivante :

- Les valeurs des noeuds sont stockés dans un **tableau**.
- La racine de l'arbre est stockée en position $0$.
- Les fils gauche et droit du noeud d'indice $i$ sont respectivement stockés aux indices $2i + 1$ et $2i + 2$.
- Le parent du noeud d'indice $i$ est le noeud d'indice $(i - 1) \ / \ 2$.

1. Remplissez les noeuds de l'arbre suivant, lorsque le tableau qui le représente est `{ 12, 9, 7, 8, 7, 5, 4, 2, 6 }`.




> Un ***tas binaire***, ou simplement *tas* (en anglais *heap*), est un arbre binaire, tel que :
>
> - L'arbre est *presque complet* : si l'arbre est de hauteur $h$, tous les niveaux de profondeur $0$ à $h - 1$ sont complets (ces niveaux contiennent $2^h$ noeuds), et les noeuds de profondeur $h$ sont le plus à gauche possible.
> - Pour tout noeud de l'arbre, la valeur du noeud est supérieure ou égale à la valeur de ses fils.

L'arbre de la question 1 est un exemple de tas.

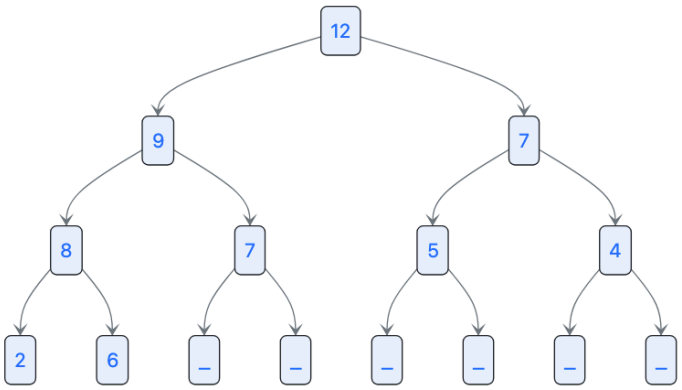
2. Quelle est la valeur de la hauteur d'un arbre complet contenant $n$ noeuds ?

```
La hauteur d'un arbre complet est log_2(n), soit O(log n).
```

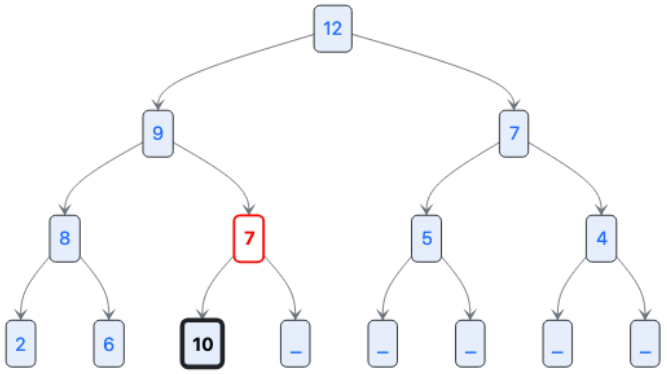
L'algorithme d'**ajout dans un tas** est le suivant. On souhaite ajouter la valeur $x$ au tas.

- Ecrire la valeur $x$ à la suite des autres valeurs, en position $n$ si l'arbre contient déjà $n$ valeurs.
- Tant que $x$ n'est pas la racine et que la valeur de $x$ est strictement supérieure à celle de son parent, échanger $x$ et son parent.

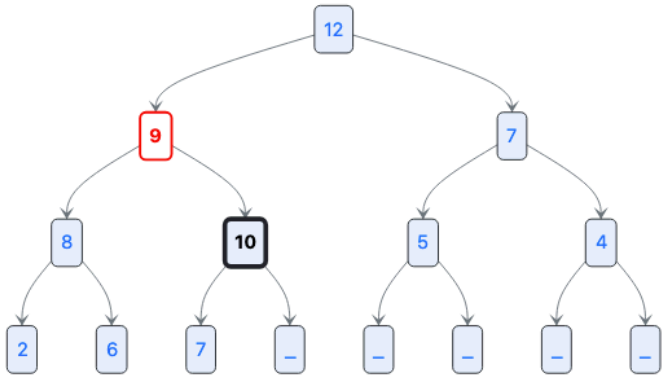
**Exemple** : On souhaite insérer la valeur $10$ dans le tas suivant :




- **1)** On insère $10$ le plus en bas à droite possible, de manière à conserver un arbre presque complet.
- **2)** On inverse le noeud $10$ avec son parent $7$, car $10 > 7$.

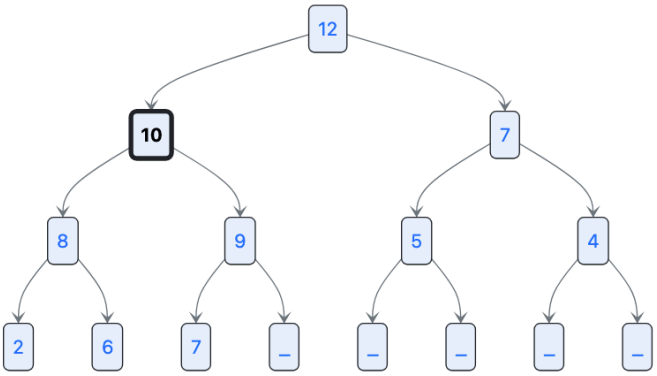



1) Insertion de **10** dans l'arbre.




2) Après échange de **10** et **7**

- **3)** On inverse de nouveau le noeud contenant $10$ avec son parent, car $10 > 9$.




3) Après échange de **10** et **9**, l'arbre est de nouveau un tas

On a terminé l'opération d'insertion, car $10 < 12$. L'arbre est bien un tas.

3. Implémentez la fonction `add_value_to_heap` qui ajoute la valeur `value` au tas.
   On pourra utiliser la fonction `swap(t, i, j)`, qui échange les éléments `t[i]` et `t[j]` du tableau `t`.

```
void add_value_to_heap(int heap[], int* nb_values, int value) {

    int i = *nb_values;
    heap[i] = value;
    (*nb_values)++;

    if (i == 0)
        return;

    int parent_index = (i - 1) / 2;

    while (i > 0 && heap[parent_index] < value) {
        swap(heap, i, parent_index);
        i = parent_index;
        parent_index = (i - 1) / 2;
    }
}
```

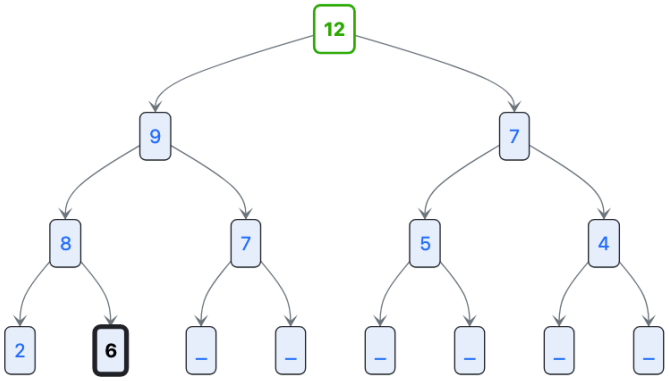
4. Quel est le coût de l'opération d'insertion, en fonction du nombre $n$ de valeurs présentes dans le tas ? Justifiez votre réponse.

```
L'opération d'insertion réalise des comparaisons et des échanges le long d'un
chemin entre une feuille et la racine. Sa complexité en pire cas est donc en
O(h), où h est la hauteur de l'arbre.
Cette hauteur vaut h = O(log n) dans un tas, donc l'opération a un coût O(log
n) en pire cas.
```

L'algorithme d'**extraction du maximum d'un tas** est le suivant :

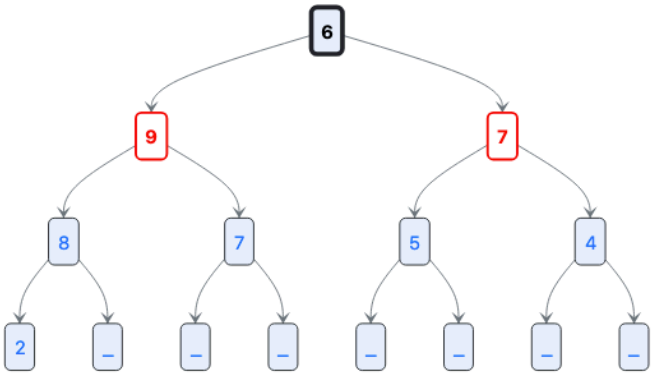
- Mémoriser la valeur $x_0$ de la racine
- Soit $x$ la valeur en position $n-1$ (si le tas contient $n$ éléments). Ecrire $x$ à la position $0$.
- **Entassement de $x$** :
  Tant que la valeur de $x$ est strictement inférieure à son fils gauche ou son fils droit :
  échanger $x$ avec son fils ayant la plus grande valeur
- Renvoyer la valeur $x_0$

**Exemple** : On souhaite supprimer le maximum du tas suivant :

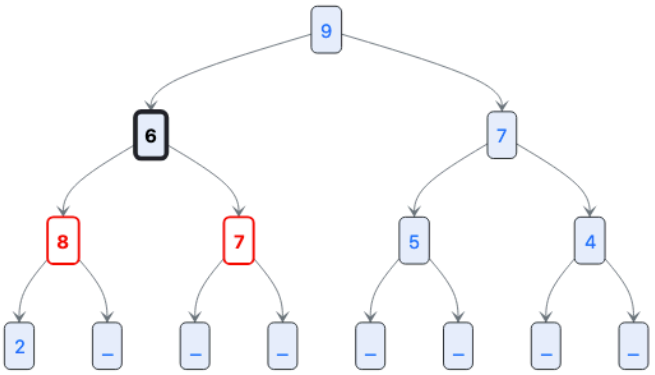



1) Etat initial du tas, le max (**12**) est à la racine

- **2)** On extrait $12$, et on le remplace par la valeur en dernière position : $6$.
- **3)** Le maximum entre $6$, $9$ et $7$ est $9$, donc on inverse $6$ et $9$.

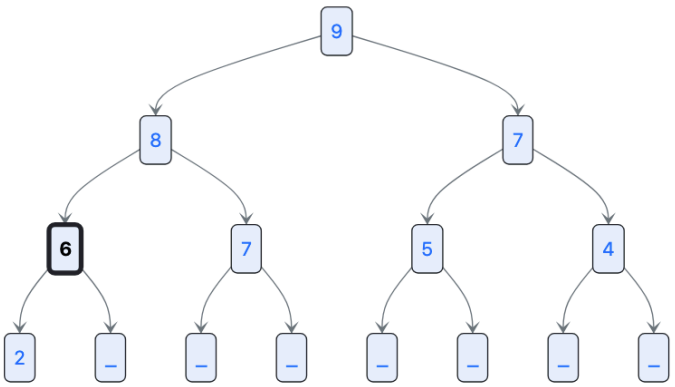



2) Après échange de **6** et **12**




3) Après échange de **6** et **9**

- **4)** Le maximum entre $6$, $8$ et $7$ est $8$, donc on inverse $6$ et $8$.




4) Après échange de **6** et **8**, l'arbre est de nouveau un tas

La valeur $6$ est supérieure à sont unique fils $2$, donc on a terminé l'opération d'extraction.

5. Implémentez la fonction `extract_max_from_heap` qui renvoie la valeur maximum du tas, et réorganise les valeurs afin de préserver la propriété de tas.

```
// Renvoie true si le noeud x d'indice i possède un fils supérieur à x
bool breaks_heap_property(int heap[], int n, int i) {

    int x = heap[i];
    int left  = 2 * i + 1;
    int right = 2 * i + 2;

    return left < n  && heap[left]  > x
        || right < n && heap[right] > x;
}

// Entassement de la valeur initialement à l'indice i
void trickle_down(int heap[], int n, int i) {

    while(  i < n        && breaks_heap_property(heap, n, i)) {

        int left  = 2 * i + 1;
        int right = 2 * i + 2;

        int max = right;
        if (heap[left] > heap[right]) {
            max = left;
        }

        swap(heap, i, max);

        i = max;

    }
}

int extract_max_from_heap(int heap[], int* n) {

    int max = heap[0];
    heap[0] = heap[(*n) - 1];
    (*n)--;
    trickle_down(heap, *n, 0);
    return max;
}
```

6. Quel est le coût de l'opération d'extraction de la valeur maximale ? Justifiez votre réponse.

```
Comme pour l'ajout, l'opération d'extraction effectue des comparaisons et des
échanges le long d'un chemin qui va d'une racine à une feuille. Son coût en
pire cas est donc en O(h) = O(log n).
```

L'algorithme du **tri par tas** est un algorithme de tri qui est défini comme le tri par file de priorité (défini en page 1), lorsque la file de priorité est implémentée par un tas.

7. Quel est le coût du tri par tas pour trier un tableau de taille $n$ ? Justifiez votre réponse.

```
L'algorithme effectue n insertions de coût O(log n), et n extractions de coût
O(log n). On a donc un coût total en O(n log n).
```

## Bonus

On propose une stratégie alternative de construction d'un tas à partir des $n$ valeurs d'un tableau $T$ :

- On place les $n$ valeurs dans un tableau dans le même ordre que dans le tableau $T$.
- On applique l'opération d'**entassement** au noeud d'indice $i$ pour $i$ de $n-1$ à $0$ dans cet ordre. (L'opération d'entassement est décrite en page 5 au sein de l'opération d'extraction.)

8. Quelle est la complexité de cette opération de construction à partir d'un tableau de $n$ éléments ?

   Indication : On donne la propriété : $\sum_{k=0}^{\infty} k2^{-k} = 2$

On suppose pour simplifier que $n = 2^h - 1$ pour un certain $h$, qui est la hauteur de l'arbre.

Les entassements des noeuds du dernier étage de l'arbre s'appliquent à des sous-arbres contenant un seul noeud, ils peuvent donc être ignorés.

Pour chaque noeud de l'étage de l'arbre de profondeur $k$, l'entassement s'applique à un sous-arbre de hauteur $h-k$, et a donc un coût $h-k$. Il y a $2^k$ noeuds à cet étage.

Le coût total est donc $\sum_{k=0}^{h-1}(h-k)2^k = \sum_{k=1}^{h} k2^{h-k} = 2^h \sum_{k=1}^{h} k2^{-k} < 2^h \sum_{k=0}^{\infty} k2^{-k} = 2(n+1) = O(n).$

Cette construction du tas s'effectue en temps $O(n)$, ce qui est plus efficace asymptotiquement que les $n$ insertions que l'on réalisaient jusque-là en temps $O(n \log n)$.